

Viking GenAI Automation Framework - Working Guide

Viking GenAI Automation Framework - Working Guide

Overview

This guide explains the end-to-end behavior of the Viking GenAI Automation application. It

cover

Prerequisites

- Python 3.9 or higher

- Node

Installation

Backend Setup

1. Open a terminal in the repository root.

2. Ins

Frontend Setup

1. Navigate to the `frontend` directory:

```b

### Application Startup

## Unified Runner

From the repository root, start the application using:

```
python run_app.py
```

What `run\_app.py` does:

- checks and verifies Python dependencies

- insta

## Manual Startup

If required, start the backend manually from the root:

```
uvicorn backend.main:app --reload --host 0.0.0.0 --port 8000
```

And the frontend manually from `frontend`:

```
npm run dev
```

## Application Architecture

The application consists of:

- Frontend: React + Vite user interface

- Bac

## User Interface Behavior

### Setup Tab

- Upload a prompt file in `.csv`, `.xls`, or `.xlsx` format.

- The

## Mode Selection

- **Document Based**: Compares model output against an expected response using semantic

score

## Configuration Modal

- Enter and save the Viking API Key or bearer token.

- Confirm

## Execution Flow

- Click **RUN** to submit the prompt file and selected mode.

- The

## Reporting and Export

- Export controls use green background and black text for high visibility.

- XLS

## Backend Behavior

### API Endpoints

- `GET /api/config`` ? returns persisted app config

- `POST`

## Evaluation Request Handling

The `/api/evaluate`` endpoint accepts:

- ``prompt_file``: uploaded CSV/XLS/XLSX file

- ``exp``

## File Parsing and Normalization

- CSV files are parsed with ``pandas.read_csv``

- Exc

## Document Mode Behavior

- Prompts are sent to the Viking API

- Exp

## RAG Mode Behavior

- Validates provider, endpoint URL and index name

- Use

## Viking API Integration

``backend/app/viking_api.py`` manages:

- bearer token authentication and refresh logic

- fallback

Evaluation Engine

- The Evaluator uses Gemini through DeepEval to compute:

- SC

Execution History

- The backend stores the last 100 session summaries in memory

- Eac

Configuration Persistence

Config Manager

Stored in `backend/config.json`, the config manager:

- initializes default values if missing

- mer

Default Config Targets

- Viking base URL and endpoint mapping

- RAC

Export and Reporting

XLSX Export

- Creates `Detailed Report` sheet with every result row

- Flat

## PDF Export

- Generates an A4 landscape report with:

- ex

## Runtime Notes

- The startup script handles dependency checks and cross-platform compatibility

- The

## Troubleshooting

- **Frontend fails to start**: re-run ``npm install`` in ``frontend``

- **Ba**

## Presentation Notes

- The app is designed to test AI prompt responses against both normal expected answers and

retri